CONFIGURAÇÃO DE REDE: IP, NETWORKMANAGER E ARQUIVOS

*O servidor precisava de IP fixo. Dois comandos — `ip addr add` e `ip route add` — e funcionou na hora. Depois da queda de energia, o IP voltou a ser o do DHCP; alguém anotou os comandos num arquivo "rodar isso depois de todo boot". **Esse arquivo é o sintoma:** configurar e persistir são operações separadas.*

079
GUIAS LINUX

QUEM REALMENTE MANDA NA INTERFACE — E POR QUE A DE CIMA SEMPRE VENCE

GERENCIADOR DE REDE
`NetworkManager · systemd-networkd · ifupdown`

Persistente. Lê perfis em disco e os **reaplica**: no boot, ao conectar o cabo, ao renovar o DHCP, ao acordar da suspensão. No Kali é o NetworkManager: um `.nmconnection` por perfil em `/etc/NetworkManager/system-connections/`, modo `600`.

↓ aplica no boot e a cada evento de link

COMANDO IP
altera o estado agora, não grava nada

Efêmero. Escreve direto no kernel. Some no reboot — e some **antes disso**, na próxima vez que o gerenciador agir sobre a interface. Lugar legítimo: teste, diagnóstico, recuperação, contêiner sem gerenciador.

↓ netlink

KERNEL
`/proc/net · /sys/class/net`

O estado real — é o que `ip addr` lê. Não guarda **intenção**: não sabe dizer se um endereço veio de perfil, de DHCP ou da sua mão. Por isso "quem configurou isto?" não se responde olhando o kernel.

A regra que evita o problema: configurou com `ip` numa máquina com gerenciador ativo? A configuração **tem prazo de validade** — ou marque a interface como não gerenciada, ou mude pelo gerenciador. Quem manda sai de `nmcli device status`: `connected` = o NM assumiu e reaplica o perfil quando quiser; `unmanaged` = e só aí o manual sobrevive.

INTERFACES VIRTUAIS: TODAS NASCEM DE `IP LINK ADD`, TODAS SÃO EFÊMERAS

BRIDGE




```
ip link add br0 type bridge
```

Switch em software: placa física e VMs no mesmo segmento.

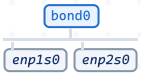
VLAN




```
ip link add link enp0s3 name
enp0s3.10 type vlan id 10
```

Uma placa, várias redes lógicas marcadas com 802.1Q.

BOND




```
ip link add bond0 type bond mode
active-backup
```

Duas placas, um endereço: redundância ou soma de banda.

VETH + NETNS




```
ip netns add lab; ip link set veth1
netns lab
```

Pilha de rede própria — rotas e firewall separados. É o que o Docker usa por baixo.

Todas somem no reboot — para persistir, `nmcli connection add type bridge|vlan|bond`. Faltam `dummy` e `tun/tap` (VPN).

NA PRÁTICA: O IP FIXO QUE SOBREVIVE AO BOOT

kali@kali: ~

```
$ nmcli device status
enp0s3  ethernet  connected  Wired connection 1
$ sudo nmcli con add type ethernet con-name lab-fixo \
    ifname enp0s3 ipv4.method manual ipv4.gateway 10.0.0.1 \
    ipv4.addresses 10.0.0.50/24 ipv4.dns "1.1.1.1 9.9.9.9" \
    ipv6.method disabled connection.autoconnect yes
$ sudo nmcli con up lab-fixo   # só agora vale
$ sudo nmcli con mod lab-fixo ipv4.method auto \
    ipv4.gateway "" ipv4.addresses "" ipv4.dns ""   # "" APAGA
$ ls -l /etc/resolv.conf   # quem regrava o DNS?
... -> /run/systemd/resolve/stub-resolv.conf
# 127.0.0.53 = stub do resolved: servidor real em `resolvectl
# status`, e POR INTERFACE — daí a VPN quebrar só alguns nomes
```

OS CINCO DETALHES QUE MORDEM

1. **`ip addr add` soma, não troca** — use `ip addr replace`
2. **Máscara não é opcional:** sem `/24` vira `/32` e a máquina não fala nem com o vizinho
3. **`ipv4.method manual` substitui** o DHCP; com `auto` o endereço é somado ao dele
4. **`connection.autoconnect yes`** faz subir no boot — sem ele o perfil existe e "não pega"
5. editar o `.nmconnection` à mão exige **`con reload`**

NET-TOOLS → IPROUTE2

`ifconfig · route -n`	`ip addr · ip route`
`arp -a · netstat -tulpn`	`ip neigh · ss -tulpn`

Mover o IP da placa para uma bridge derruba a conexão no instante em que o endereço sai da placa — numa máquina remota, é ficar trancado do lado de fora. Faça pelo console, ou num script com rollback agendado por `at`. Idem `ip addr flush dev enp0s3`, que remove **todos** os endereços e mata o SSH em curso; e `chattr +i /etc/resolv.conf`, que funciona mas **quebra o gerenciador em silêncio**.

RESUMO RÁPIDO

- ✓ **Configurar e persistir são operações distintas:** `ip` fala com o kernel e é efêmero; persistir é do gerenciador.
- ✓ **Com NetworkManager ativo, o manual tem prazo** — e o prazo não é o reboot, é o próximo toque na interface.
- ✓ **`ip addr add` soma e IP sem máscara vira `/32`:** os dois erros mais frequentes.
- ✓ **`ipv4.method manual` + `autoconnect yes`** faz o IP fixo pegar e subir no boot.
- ✓ **NM e `ifupdown` na mesma interface** = máquina que se reconfigura sozinha, com cara de defeito de hardware.

NO KALI

Namespace de rede é o laboratório mais barato que existe. Dois `ip netns` ligados por um par `veth` reproduzem cliente, servidor e roteador numa máquina só, com firewall independente, sem VM e **sem risco de tocar em rede alheia** — o ambiente certo para praticar firewall e captura. **Modo monitor não é configuração de rede:** a placa perde IP e o NM tenta reassumi-la — daí o `airmon-ng check kill`; depois, `airmon-ng stop wlan0mon` e `systemctl restart NetworkManager` devolvem o normal. **MAC aleatório é recurso do próprio NM** (`wifi.cloned-mac-address random|stable|permanent`): privacidade legítima em Wi-Fi público — usá-lo para burlar controle de acesso alheio é acesso não autorizado (Lei 12.737/2012). **Perfil guarda segredo em claro:** os `.nmconnection` têm PSK de Wi-Fi e credencial de VPN protegidos só pelo modo `600` — em máquina emprestada, entram na limpeza junto com `~/.ssh`. **RHEL:** o 9 removeu os `ifcfg-*`; script antigo falha em silêncio.